

Databases 2 - exam - July 2, 2025 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Concurrency control (12 points)

Consider the following schedule:

$$S = r_1(X) w_2(X) w_3(Z) w_1(X) w_5(X) r_6(Z) r_5(Y) r_3(Y) r_3(Z) r_5(X) r_2(Y) w_5(Y) r_5(Z)$$

Establish membership or non-membership of S with respect to the serializability classes VSR, CSR, 2PL, TS Mono, TS Mono with Thomas Rule, TS Multi (with the usual conventions adopted in practice). Justify your answers with terse but cogent explanations. Solutions should use serializability class inclusion whenever possible.

Solution.

Summary

S is VSR.

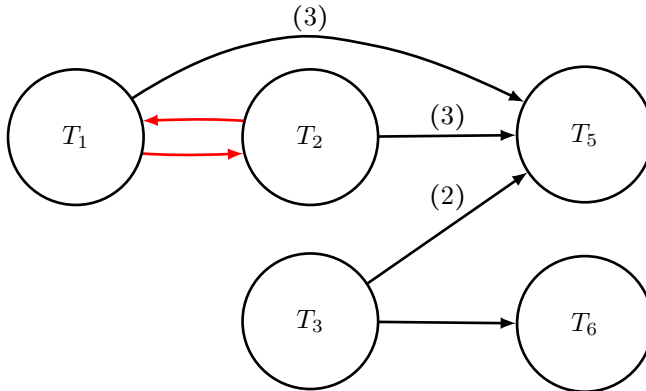
S is not CSR, and thus also not 2PL, and not TS-Mono.

S is not TS-Multi (and thus also not TS-Mono).

S is TS with Thomas Rule.

Detailed analysis for the CSR class

The resource graph is cyclic (cycles in red).



Detailed analysis for the VSR class

Although S is not CSR, it becomes CSR by swapping the blind writes w_2 and w_1 on X .

Detailed analysis for the TS Thomas class

Operation	RTM(X)	WTM(X)	RTM(Y)	WTM(Y)	RTM(Z)	WTM(Z)	Killed transactions
$r_1(X)$	1 (0)	0	0 (0)	0	0 (0)	0	
$w_2(X)$		2					
$w_3(Z)$						3	
$w_1(X)$							skipped
$w_5(X)$		5					
$r_6(Z)$					6 (0)		
$r_5(Y)$			5 (0)				
$r_3(Y)$			5 (0)				
$r_3(Z)$					6 (0)		
$r_5(X)$	5 (0)						
$r_2(Y)$			5 (0)				
$w_5(Y)$				5			
$r_5(Z)$					6 (0)		

Detailed analysis for the TS Multi class

Operation	RTM(X)	WTM(X)	RTM(Y)	WTM(Y)	RTM(Z)	WTM(Z)	Killed transactions
$r_1(X)$	1 (0)	0	0 (0)	0	0 (0)	0	
$w_2(X)$		0, 2					
$w_3(Z)$						0, 3	
$w_1(X)$							1 as $1 < \text{last WTM}(X) = 2$
$w_5(X)$		0, 2, 5					
$r_6(Z)$					6 (3)		
$r_5(Y)$			5 (0)				
$r_3(Y)$			5 (0)				
$r_3(Z)$					6 (3)		
$r_5(X)$	5 (5)						
$r_2(Y)$			5 (0)				
$w_5(Y)$				0, 5			
$r_5(Z)$					6 (3)		

B. Ranking (10 points)

In the classical “CPU” dataset used for Machine Learning (ML) tasks, CPU data are reported, focusing on performance indicators, among which MYCT (machine cycle time in ns: lower is better), MMIN (minimum main memory in KiB: higher is better), MMAX (maximum main memory in KiB: higher is better), CACH (cache memory in KiB: higher is better). An ML algorithm determines the following function for estimating the relative performance of the CPU: $erp(t) = -100t.MYCT + 10t.MMIN + t.MMAX + t.CACH$.

MYCT	MMIN	MMAX	CACH
g: 25	b: 8000	b: 32000	a: 256
b: 29	d: 2000	d: 8000	g: 131
h: 50	g: 1310	a: 6000	d: 65
d: 60	c: 1000	h: 4000	b: 32
a: 125	h: 1000	c: 3000	f: 8
f: 167	f: 524	g: 2620	h: 8
e: 350	a: 256	f: 2000	c: 0
c: 400	e: 64	e: 64	e: 0

1. If possible, apply TA (or adapt it appropriately) to determine the top-1 CPU in the dataset shown on the side, where values are sorted from best to worst; if not, explain why. You should indicate the **depth**, the number of **accesses**, the **results**, and succinctly show enough details so as to allow understanding each round of execution as well as the **stopping criterion**.
2. Show the skyline of the dataset. You don't have to show any steps used to compute it.

Solution.

1. TA can be applied even though the scoring function is, in principle, not monotone because of the “-” sign in front of the 100MYCT component. Indeed, MYCT values are listed in increasing order, so that later values reduce the *erp* performance, as happens with the other attributes.

ROUND 1

```
s.a. to MYCT: g (25.0)
r.a. to MMIN: g, score=1310.0
r.a. to MMAX: g, score=2620.0
r.a. to CACH: g, score=131.0
s.a. to MMIN: b (8000.0)
r.a. to MYCT: b, score=29.0
r.a. to MMAX: b, score=32000.0
r.a. to CACH: b, score=32.0
s.a. to MMAX: b (32000.0)
s.a. to CACH: a (256.0)
r.a. to MYCT: a, score=125.0
r.a. to MMIN: a, score=256.0
r.a. to MMAX: a, score=6000.0
Buffer:
b, 29, 8000, 32000, 32, score: 109132
T, 25, 8000, 32000, 256, score: 109756
```

ROUND 2

```
s.a. to MYCT: b (29.0)
s.a. to MMIN: d (2000.0)
r.a. to MYCT: d, score=60.0
r.a. to MMAX: d, score=8000.0
r.a. to CACH: d, score=65.0
s.a. to MMAX: d (8000.0)
s.a. to CACH: g (131.0)
Result:
b, 29, 8000, 32000, 32, score: 109132
T, 29, 2000, 8000, 131, score: 25231
```

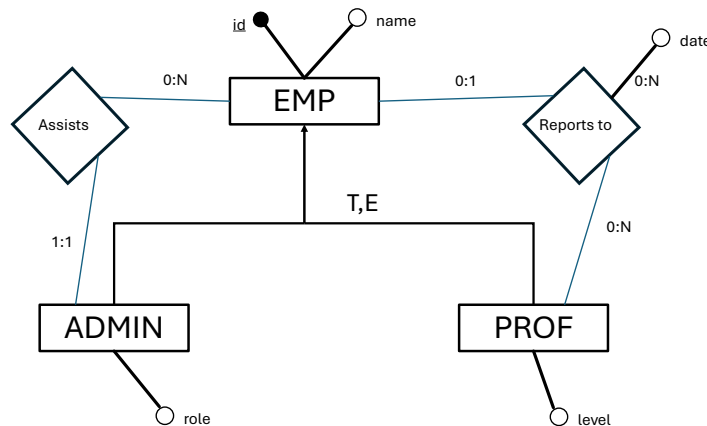
depth: 2, sorted: 8; random: 12

CPU *b* is the top-1 result

2. The skyline is $\{g, b, d, a\}$.

C. Trigger (10 points)

Consider the following ER schema, in which employees are either administrative staff or professors. An admin assists exactly one employee, and an employee may have to report to a supervising professor (possibly, on multiple dates). All other attributes (`id`, `name`, `role`, `level`) are mandatory in their entities.



In the following logical schema, all the entities (as well as `Assists`) collapse to `EMP`, where the `*` indicates that the attribute is nullable and `isProf` is a boolean selector that is true iff the entity is a professor:

`EMP(ID, isProf, name, supervisor*, role*, assistedEmp*, level*)`

`ReportsTo(EmpID, date)`

Not all constraints implicit in the ER schema are captured by the logical schema. Write a trigger system for enforcing coherence of this representation, so that the `isProf` selector synchronises correctly with the attributes and foreign keys of the `ADMIN` and `PROF` entities. In particular:

1. professors must have a `level` and not a `role`; the opposite holds for admins;
2. the `supervisor` foreign key must reference instances of `PROF`;
3. a tuple in `ReportsTo` must reference, through `EmpID`, a tuple in `EMP` whose `supervisor` is not null.

You must handle insertions and deletions for both relations (but not update events). You can manage some of the constraints in the table definition, but if you do so please include the table creation statements in your solution. Before writing the code, list all the constraint violations that need to be prevented and specify how you handle each one of them (trigger or table creation).

Solution.

Deletions are managed via referential integrity constraints specified at table creation with `ON DELETE` constructs. Point 1 is handled through a `CHECK` constraint in the `EMP` table. Points 2 and 3 are handled through triggers.

```
CREATE TABLE EMP (  
  ID          INT NOT NULL PRIMARY KEY,  
  isProf      BOOLEAN NOT NULL,  
  name        VARCHAR(30) NOT NULL,  
  supervisor  INT REFERENCES EMP(ID) ON DELETE SET NULL, -- insertion managed through triggers  
  role        CHAR(15),  
  assistedEmp INT REFERENCES EMP(ID) ON DELETE NO ACTION, -- deletion disallowed  
  level       INT,  
  CONSTRAINT SYNC CHECK (  
    (isProf IS TRUE AND level IS NOT NULL AND role IS NULL) OR  
    (isProf IS FALSE AND level IS NULL AND role IS NOT NULL AND assistedEmp IS NOT NULL))  
);  
  
CREATE TABLE ReportsTo (  
  EmpID INT NOT NULL REFERENCES EMP(ID) ON DELETE CASCADE,  
  date  DATE NOT NULL,  
  PRIMARY KEY (EmpID,date) );
```

```

-- Trigger for the supervisor FK
CREATE OR REPLACE TRIGGER supervisorFK
BEFORE INSERT ON EMP
REFERENCING NEW AS N
FOR EACH ROW
WHEN ( EXISTS ( SELECT *
                FROM EMP
                WHERE N.supervisor = EMP.ID
                AND EMP.isProf IS FALSE ) )
SIGNAL SQLSTATE '70001' SET MESSAGE_TEXT = 'The supervisor is not a Prof!';

-- Trigger checking existence of supervisor
CREATE OR REPLACE TRIGGER validReports
BEFORE INSERT ON ReportsTo
REFERENCING NEW AS N
FOR EACH ROW
WHEN ( NOT EXISTS ( SELECT *
                   FROM EMP
                   WHERE N.EmpID = EMP.ID
                   AND EMP.supervisor IS NOT NULL ) )
SIGNAL SQLSTATE '70002' SET MESSAGE_TEXT = 'The employee has no supervisor!';

```